Design and Implementation of a Cascaded PID Controller for a Non-linear Quadrotor Under Stochastic Disturbances

Henry Rochester

Supervisor: Dr Malcolm Hillebrand

Department of Mathematics and Applied Mathematics
University of Cape Town

April 12, 2026

Introduction

We now implement a two-layer cascaded PID controller for the quadrotor. The objective of this controller is to drive the drone from an initial position to a desired target position and maintain hover at that target, even in the presence of stochastic disturbances.

A PID controller is a feedback controller which adjusts the control input according to the error between the current process variable and the desired target (also known as setpoint). The control action is composed of three terms: a proportional term, an integral term, and a derivative term.

Control Terminology

We define the following control-theoretic terms for clarity:

- Process Variable (PV): The system output $y(t)$ that is being measured and controlled.
- Setpoint (Target): The desired reference value $r(t)$ for the process variable.
- Control Variable: The controller output $C(t)$ which is applied to the system in order to minimise the error.
- Steady-State Value: The limiting value of the process variable as $t \to \infty$, denoted $\lim_{t\to\infty} y(t)$.
- Steady-State Error: The residual error at steady state, given by:

$$e_{\text{ss}} = \lim_{t\to\infty}(r(t) - y(t)).$$

General PID Structure

Let $r(t)$ denote the setpoint and let $y(t)$ denote the process variable. We define the tracking error by the following expression:

$$e(t) = r(t) - y(t).$$

We note that the continuous-time PID control law is then given by:

$$C(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de}{dt}(t),$$

where:

- $C(t)$ is the control variable;
- K_p is the proportional gain;
- K_i is the integral gain;
- K_d is the derivative gain.

The proportional term responds to the present error; the integral term accumulates past error so as to reduce steady-state error, and the derivative term reacts to the rate of change of the error, thereby introducing damping and reducing oscillation.

In implementation, the derivative term is approximated numerically by:

$$\frac{de}{dt}(t) \approx \frac{e_k - e_{k-1}}{\Delta t},$$

where e_k is the current time-step error, e_{k-1} is the error at the previous time step, and Δt is the simulation time step.

Similarly, the integral term is approximated discretely by:

$$I_k = I_{k-1} + e_k \Delta t.$$

Thus, at each time step, the implemented PID control law takes the following form:

$$C_k = K_p e_k + K_i I_k + K_d \frac{e_k - e_{k-1}}{\Delta t}.$$

Controller Architecture for the Quadrotor

For the quadrotor, we employ six individual PID sub-controllers:

- one controller for vertical position z;
- one controller for horizontal position x;
- one controller for horizontal position y;
- one controller for roll angle ϕ;
- one controller for pitch angle θ;
- one controller for yaw angle ψ.

These controllers are arranged in a two-layer cascaded structure.

Outer Loop: Position Control

The outer loop is responsible for translational motion. It takes the position errors in x, y, and z and converts them into desired thrust and desired attitude, which are then passed to the inner loop subcontrollers.

For the elevation controller, the process variable is z, the setpoint is z^*, and the error is given by:

$$e_z = z^* - z.$$

The PID controller produces a thrust correction term

$$T_{\text{corr}} = K_p^{(z)} e_z + K_i^{(z)} \int_0^t e_z(\tau)\, d\tau + K_d^{(z)} \frac{de_z}{dt}.$$

Rather than using this directly as the total thrust, we add it to the hover thrust:

$$T = mg + T_{\text{corr}}.$$

This is essential, since if the drone is exactly at the desired altitude and the controller output were set to zero, then the total thrust would vanish and the drone would fall. By adding the correction to the hover thrust mg, the controller instead adjusts thrust relative to the hover thrust required to balance gravity.

For horizontal motion, the controller converts position errors into desired attitude angles, the translational movement is coupled with the rotational dynamics of the system. Specifically, we define:

$$e_x = x^* - x, \qquad e_y = y^* - y.$$

The x-position controller outputs a desired pitch angle:

$$\theta^* = K_p^{(x)} e_x + K_i^{(x)} \int_0^t e_x(\tau)\, d\tau + K_d^{(x)} \frac{de_x}{dt},$$

and the y-position controller outputs a desired roll angle:

$$\phi^* = K_p^{(y)} e_y + K_i^{(y)} \int_0^t e_y(\tau)\, d\tau + K_d^{(y)} \frac{de_y}{dt}.$$

Thus, the outer loop transforms translational position errors into the intermediate commands:

$$T, \qquad \phi^*, \qquad \theta^*.$$

Moreover, we set $\psi^* = 0$, since the desired behaviour is for the quadrotor to maintain a

forward-facing orientation during hover.

Inner Loop: Attitude Control

We note that the inner loop is responsible for rotational motion. It takes the desired roll, pitch, and yaw angles and generates the required body torques as control variables.

For roll, pitch, and yaw respectively, we define the attitude errors as the following:

$$e_\phi = \phi^* - \phi, \qquad e_\theta = \theta^* - \theta, \qquad e_\psi = \psi^* - \psi.$$

The corresponding PID controllers generate the torque commands:

$$\tau_x = K_p^{(\phi)} e_\phi + K_i^{(\phi)} \int_0^t e_\phi(\tau)\, d\tau + K_d^{(\phi)} \frac{de_\phi}{dt},$$
$$\tau_y = K_p^{(\theta)} e_\theta + K_i^{(\theta)} \int_0^t e_\theta(\tau)\, d\tau + K_d^{(\theta)} \frac{de_\theta}{dt},$$
$$\tau_z = K_p^{(\psi)} e_\psi + K_i^{(\psi)} \int_0^t e_\psi(\tau)\, d\tau + K_d^{(\psi)} \frac{de_\psi}{dt}.$$

The final control input to the quadrotor is therefore

$$\mathbf{u} = \begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}.$$

We note that in the physical system, these total thrust and body torque commands would subsequently need to be converted into individual rotor speeds, which are then applied through the electronic speed controllers and motors.

Interpretation of the PID Terms

Each PID term plays a distinct role in the control response:

- The proportional term, $K_p e(t)$, provides an immediate response to the present error. A larger error produces a larger control action, and vice-versa.
- The integral term, $K_i I_k$, accumulates past error and is primarily responsible for removing steady-state error.
- The derivative term, $K_d \frac{e_k - e_{k-1}}{\Delta t}$, introduces damping by responding to the rate at which the error is changing, thereby reducing overshooting and oscillation within the system.

We note that if only the proportional control were used, then the drone would typically approach the setpoint but may settle with a non-zero steady-state error, as the drone asymptotically approaches the setpoint. The integral term corrects this by accumulating residual error over time. However, this introduces the risk of *integral windup*, where the integral term becomes excessively large when the actuator is saturated.

To mitigate this, we impose an integral clipping condition of the form:

$$I_k \in [-I_{\max}, I_{\max}],$$

where $I_{\max}$ is a prescribed integral limit. This prevents the controller from accumulating an unrealistically large integral contribution when the physical system is unable to realise further control input.

An important note is that we employ a cascaded controller because translational motion and rotational motion occur on different physical levels. We recall that in order for the drone to move forward, backward, left, or right, it must first tilt (pitch, roll, respectively) appropriately. Thus, the outer loop determines what attitude is required to achieve the desired position change, while the inner loop ensures that this attitude is achieved.

It follows that the inner attitude loop must therefore be tuned first, since the outer position loop assumes that desired angles can be tracked reliably. If the inner loop is unstable or too slow, then the outer loop will not function correctly.

Gain Tuning Procedure

To tune the six PID controllers, we adopt a practical variation of the Ziegler-Nichols closed-loop tuning procedure for non-linear systems.

We begin with the inner loop, as mentioned above. The tuning is performed one controller at a time.

We demonstrate by means of an example, say for the roll controller:

1. Set $K_p = 0$,$K_i = 0$, $K_d = 0$.
2. Introduce a small roll perturbation, for example $\phi(0) = 0.1$ rad.

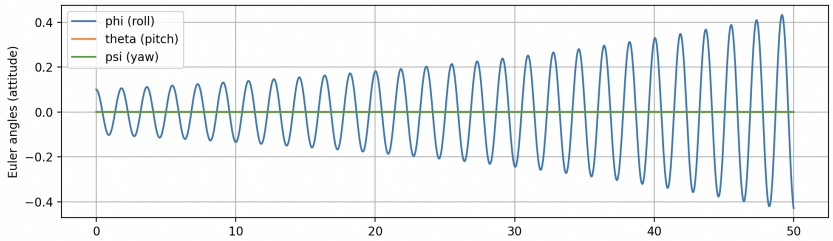

Figure 1: Response under initial roll perturbation.

3. Increase K_p gradually until the response exhibits sustained oscillation. This corresponds to the edge of stability for the control variable.

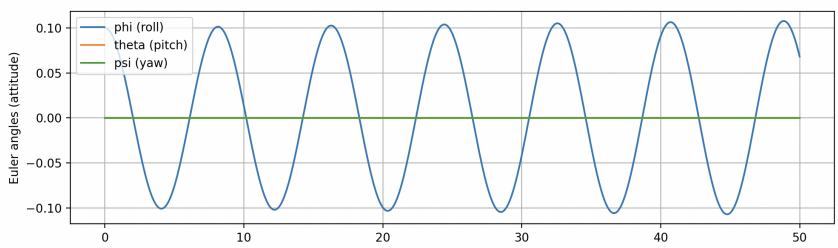

Figure 2: Sustained oscillations at critical proportional gain.

4. Fix K_p and then increase K_d until the oscillation is sufficiently damped, ideally yielding a single oscillation before convergence.

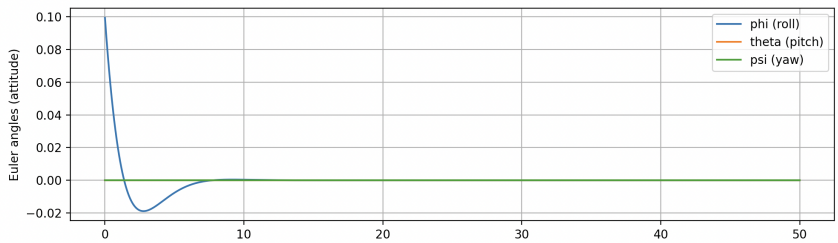

Figure 3: Damped response after tuning derivative gain.

5. Finally, we increase K_i slowly until the steady-state error is removed and the response settles at the setpoint (in this example $K_i = 0$ since graph above already settles at setpoint).

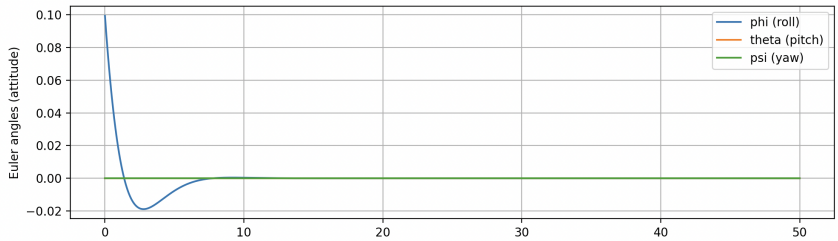

Figure 4: Final response after integral tuning with zero steady-state error.

Once the roll controller has been tuned, we reset the small perturbation ϕ to zero and repeat the same procedure for pitch θ, yaw ψ, and then for the outer-loop controllers z, x, and y.

Finally, we note that the present cascaded controller is designed for the hover and point-to-point stabilisation setting. In this case, the desired terminal velocity is zero, and the position error dynamics are sufficiently captured by the two-layer cascade.

In particular, the velocity $\vec{v}(t)$ and angular velocity $\vec{\omega}(t)$ are implicitly accounted for through the derivative terms in the PID controllers. Indeed, for position tracking, we have:

$$\dot{e}(t) = \frac{d}{dt}(r(t) - y(t)) = -\dot{y}(t),$$

since the setpoint $r(t)$ is constant in the hover setting. Thus, the derivative term provides direct feedback of the system velocity.

In discrete time, this is approximated by

$$\frac{e_k - e_{k-1}}{\Delta t} \approx -v,$$

for translational motion, and similarly

$$\frac{e_k - e_{k-1}}{\Delta t} \approx -\omega,$$

for rotational motion.

Hence, the velocity and angular velocity dynamics are naturally incorporated within the two-layer PID structure, and no additional control layer is required for the hover scenario.

For more advanced tasks such as trajectory tracking or target following, where the desired velocity is non-zero, this two-layer structure is generally insufficient. In such cases, we would need to introduce an additional velocity-control layer, yielding a deeper cascade in which position generates velocity outputs, velocity generates attitude outputs,

and attitude generates torque outputs.

References

1. B. Douglas, *PID Control MATLAB Playlist*, YouTube. Available at: PID Control Playlist (Accessed: April 2026)

2. N. H. Sahrir and M. A. M. Basri, *Modelling and Manual Tuning PID Control of Quadcopter*, School of Electrical Engineering, Faculty of Engineering, Universiti Teknologi Malaysia, Johor Bahru, Malaysia, 2022

3. A. L. Salih, M. Moghavvemi, H. A. F. Mohamed, and K. S. Gaeid, *Flight PID Controller Design for a UAV Quadrotor*, Centre for Research in Applied Electronics (CRAE), University of Malaya, Kuala Lumpur, Malaysia, 2010.

4. Duckietown, *PID Controllers: Generalities*, Duckietown Documentation. Available at: Duckietown PID Controllers Page (Accessed: April 2026).